Migration SQL vers NoSQL et Analyse utilisant MongoDB et Cassandra

Réalisé par : Gharsalli Hind

Année scolaire : 2025 – 2026



Table des Matières

1 Introduction **3**

2 Modèle de Données Relationnel **3**

3 Migration vers JSON **4**

4 Implémentation MongoDB **5**

5 Implémentation Cassandra **6**

6 Analyse de Données **8**

7 Comparaison **9**

8 Conclusion **9**

9 Annexe **10**
9.1 Schémas 10
9.2 Captures d'écran : MongoDB 12
9.3 Captures d'écran : Cassandra 15

1 Introduction

Résumé du projet

Ce projet présente une étude comparative et pratique sur la transition entre les systèmes de gestion de bases de données traditionnels et les nouvelles architectures NoSQL.

À travers l'utilisation de **MongoDB** (orienté document) et de **Cassandra** (orienté colonnes), nous analysons comment un jeu de données relationnel structuré peut être migré, transformé et exploité pour répondre à des besoins spécifiques de flexibilité et de performance.

L'objectif est de mettre en lumière les implications directes de ces changements sur la structure des données et l'efficacité des requêtes analytiques.

2 Modèle de Données Relationnel

Objectif

Partir d'un jeu de données relationnel structuré .

Source de Données

Base de données SQL Postgresql .

Schéma

— Tables avec clés primaires, clés étrangères et structure normalisée .
— Voir `schema.sql` et `sqldata-export.sql` pour le schéma et les données .

Décisions de Conception

Les données sont relationnelles car elles sont organisées en tables avec des relations définies (entités et relations) utilisant des clés primaires et étrangères, garantissant l'intégrité des données et la normalisation .

Exemple de Schéma SQL

```sql
CREATE TABLE users (
    user_id INT PRIMARY KEY,
    name VARCHAR(100),
    email VARCHAR(100),
    city VARCHAR(50)
);

CREATE TABLE products (
    product_id INT PRIMARY KEY,
    name VARCHAR(100),
```

```
    category VARCHAR(50),
    price DECIMAL(10,2)
);

CREATE TABLE orders (
    order_id INT PRIMARY KEY,
    user_id INT,
    order_date DATE,
    FOREIGN KEY (user_id) REFERENCES users (user_id)
);

CREATE TABLE order_items (
    order_item_id INT PRIMARY KEY,
    order_id INT,
    product_id INT,
    quantity INT,
    FOREIGN KEY (order_id) REFERENCES orders (order_id),
    FOREIGN KEY (product_id) REFERENCES products (product_id)
);
```

3 Migration vers JSON

Objectif

Transformer les données relationnelles en format orienté document .

Étapes

— Extraction des données depuis SQL via un script d'export .
— Dénormalisation des données là où c'est approprié .
— Conversion des lignes en documents JSON .

Décisions de Conception

— Choix de l'imbrication (embedding) pour la performance et la simplicité lorsque cela est possible .
— Utilisation de références pour les entités volumineuses ou partagées .
— Données imbriquées pour les relations 1 :N ou fortement couplées .
— Données référencées pour les relations N :M ou faiblement couplées .
— Exemple : Jointures SQL → documents JSON imbriqués .

Livrables

— Fichiers JSON (voir `exported-data.json`) .

Requête d'Export SQL vers JSON

```
SELECT
    o.order_id,
    json_build_object(
        'user_id', u.user_id,
```

```
        'name', u.name,
        'city', u.city
    ) AS user,
    o.order_date,
    json_agg(
        json_build_object(
            'product', p.name,
            'price', p.price,
            'quantity', oi.quantity
        )
    ) AS items
FROM orders o
JOIN users u ON o.user_id = u.user_id
JOIN order_items oi ON o.order_id = oi.order_id
JOIN products p ON oi.product_id = p.product_id
GROUP BY o.order_id, u.user_id, u.name, u.city, o.order_date;
```

Document JSON (Résultat)

```
{
    "order_id": 1,
    "user": {
        "user_id": 1,
        "name": "Alice",
        "city": "Paris"
    },
    "date": "2024-01-10",
    "items": [
        {"product": "Laptop", "price": 1200.00, "quantity": 1},
        {"product": "Headphones", "price": 150.00, "quantity": 2}
    ]
}
```

4 Implémentation MongoDB

Objectif

Stocker et manipuler des données JSON dans MongoDB .

Structure

— Base de données, collections, documents .

Opérations CRUD

— insertOne, insertMany, find, update, delete .

Extraits de Code (MongoDB Shell)

```
// Inserer un document
db.orders.insertOne({
    user: { user_id: 1, name: "Alice", city: "Paris" },
    date: "2024-01-10",
    items: [
        { product: "Laptop", price: 1200.00, quantity: 1},
        { product: "Headphones", price: 150.00, quantity: 2}
    ]
})

// Inserer plusieurs documents
db.orders.insertMany([...])

// Trouver des documents
db.orders.find({ "user.city": "Paris" })

// Mettre a jour un document
db.orders.updateOne({ order_id: 1 }, { $set: { "user.city": "Lyon" } })

// Supprimer un document
db.orders.deleteOne({ order_id: 1 })
```

5 Implémentation Cassandra

Objectif

Stocker les mêmes données en utilisant un modèle orienté requête (query-driven) .

Structure

— Keyspace, tables, clés de partition, colonnes de clustering .

Opérations CRUD

— INSERT, SELECT (avec clé de partition), UPDATE, DELETE .

Schéma CQL

```
CREATE KEYSPACE IF NOT EXISTS ecommerce
WITH REPLICATION = {
    'class': 'SimpleStrategy',
    'replication_factor': 1
};

USE ecommerce;

CREATE TYPE IF NOT EXISTS user_type (
    user_id int,
    user_name text,
    city text
);

CREATE TYPE IF NOT EXISTS product_type (
```

```
    product_name text,
    price double,
    quantity int
);

CREATE TABLE IF NOT EXISTS sales_by_product (
    product text PRIMARY KEY,
    total_quantity int,
    total_revenue double,
    users set<frozen<user_type>>
);

CREATE TABLE IF NOT EXISTS sales_by_user (
    user_id int PRIMARY KEY,
    user_name text,
    total_orders int,
    total_revenue double,
    products set<frozen<product_type>>
);

CREATE TABLE IF NOT EXISTS orders_by_id(
    order_id int PRIMARY KEY,
    user_id int,
    user_name text,
    user_city text,
    date text,
    items list<frozen<tuple<text, double, int>>>
);
```

Script d'Importation de Données (Python)

```
import json
from cassandra.cluster import Cluster

# Connexion a Cassandra
cluster = Cluster(["127.0.0.1"])
session = cluster.connect("ecommerce")

# Vider toutes les tables avant l'importation
session.execute("TRUNCATE␣orders_by_id")
session.execute("TRUNCATE␣sales_by_user")
session.execute("TRUNCATE␣sales_by_product")

# Charger les donnees JSON
with open("exported-data.json") as f:
    orders = json.load(f)

# Preparer les instructions (statements)
insert_order_stmt = session.prepare("""
␣␣␣␣INSERT␣INTO␣orders_by_id␣(order_id,␣user_id,␣user_name,␣user_city,␣
    date,␣items)
␣␣␣␣VALUES␣(?,␣?,␣?,␣?,␣?,␣?)
""")

update_user_stmt = session.prepare("""
␣␣␣␣UPDATE␣sales_by_user
␣␣␣␣SET␣total_orders␣=␣total_orders␣+␣?,
```

```
        total_revenue = total_revenue + ?,
        products = products + ?
    WHERE user_id = ?
""")

update_product_stmt = session.prepare("""
    UPDATE sales_by_product
    SET total_quantity = total_quantity + ?,
        total_revenue = total_revenue + ?,
        users = users + ?
    WHERE product = ?
""")

print("Data import completed successfully.")
```

Décisions de Conception

— Modélisation axée sur les requêtes (query-first), efficacité de lecture, pas de jointures .

Livrables

— Schéma CQL (voir ci-dessus et `cassandra-schema.cql`) .
— Explication de la stratégie de partition (modélisation axée sur les requêtes, pas de jointures) .
— Exemples de requêtes .

6 Analyse de Données

Objectif

Analyser les données visuellement et statistiquement .

Utilisation de Compass

— Statistiques de collection, nombre de documents, distribution des champs .
— Pipelines d'agrégation (comptages, moyennes, regroupements, filtrage) .

Exemple de Requête d'Agrégation (MongoDB)

```
// Compter les commandes par utilisateur
db.orders.aggregate([
    { $group: { _id: "$user.user_id", total_orders: { $sum: 1 } } }
])

// Valeur moyenne des commandes
db.orders.aggregate([
    { $unwind: "$items" },
    { $group: {
        _id: "$order_id",
```

```
        total: { $sum: { $multiply: ["$items.price", "$items.quantity"] } }
    }},
    { $group: { _id: null, avg_order_value: { $avg: "$total" } } }
])
```

7 Comparaison

Comparer

— SQL vs MongoDB vs Cassandra .
— Rigidité du schéma, flexibilité des requêtes, performance, cas d'utilisation .

Tableau Récapitulatif

Fonctionnalité	**SQL**	**MongoDB**	**Cassandra**
Schéma	Rigide	Flexible	Flexible
Jointures	Supportées	Limitées	Non supportées
Modèle de Requête	Relationnel	Document	Colonnes larges (Wide-column)
Performance	ACID, jointures	Rapide, flexible	Rapide, scalable
Cas d'utilisation	OLTP, rapports	JSON, analytique	Big data, IoT

TABLE 1 – Comparaison des systèmes de base de données

8 Conclusion

— Les systèmes SQL demeurent la solution optimale pour le traitement de données relationnelles structurées nécessitant l'exécution de jointures et de requêtes complexes.
— MongoDB s'avère être l'outil de prédilection pour la gestion de données hiérarchiques, offrant des modèles à la fois hautement flexibles et évolutifs.
— Cassandra se distingue par ses performances dans la gestion de charges de travail distribuées à très grande échelle, à condition que les schémas d'interrogation soient prévisibles.
— Le processus de migration exige une attention particulière portée à la dénormalisation, ainsi qu'une modélisation structurelle entièrement guidée par les requêtes.
— Chaque technologie impliquant des compromis, l'arbitrage final doit systématiquement s'aligner sur les contraintes spécifiques et les besoins propres à chaque cas d'usage.

9 Annexe

9.1 Schémas

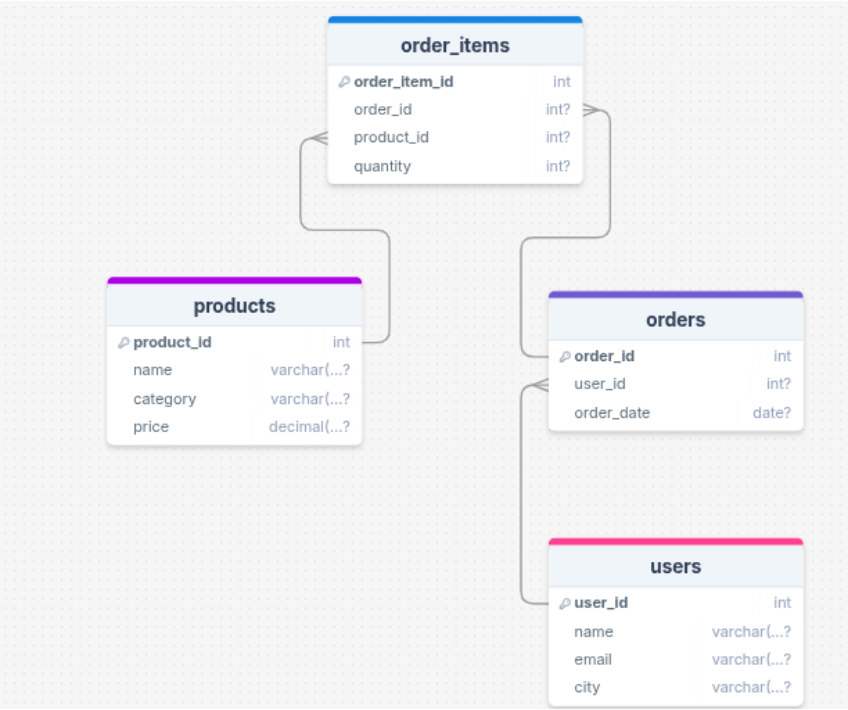


FIGURE 1 – Schéma relationnel (SQL)

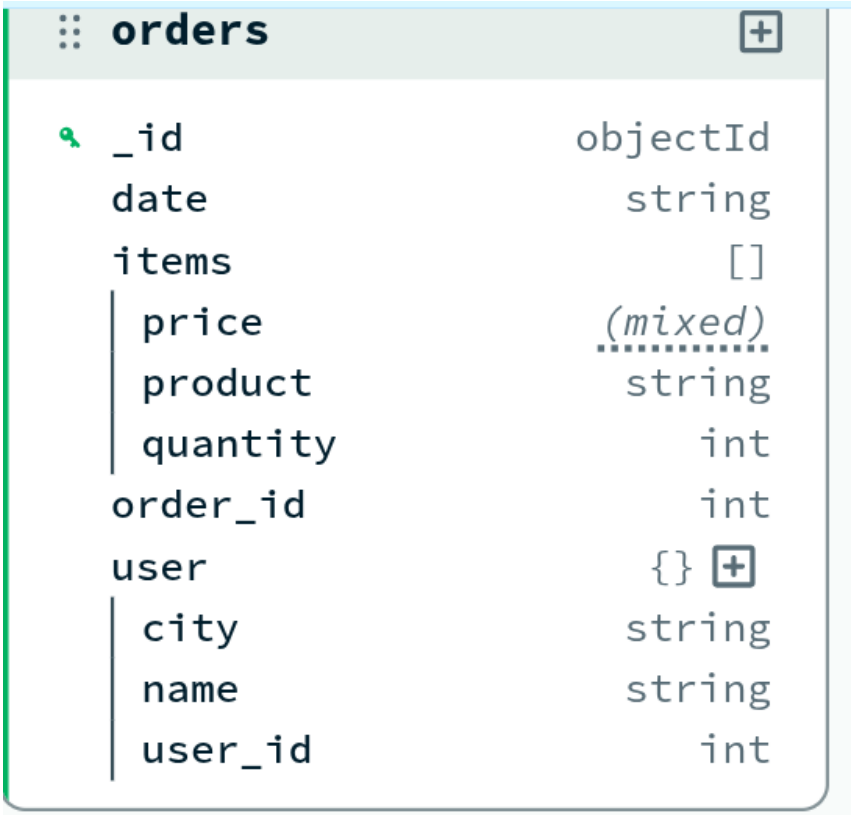


FIGURE 2 – Schéma orienté document (MongoDB)

9.2 Captures d'écran : MongoDB

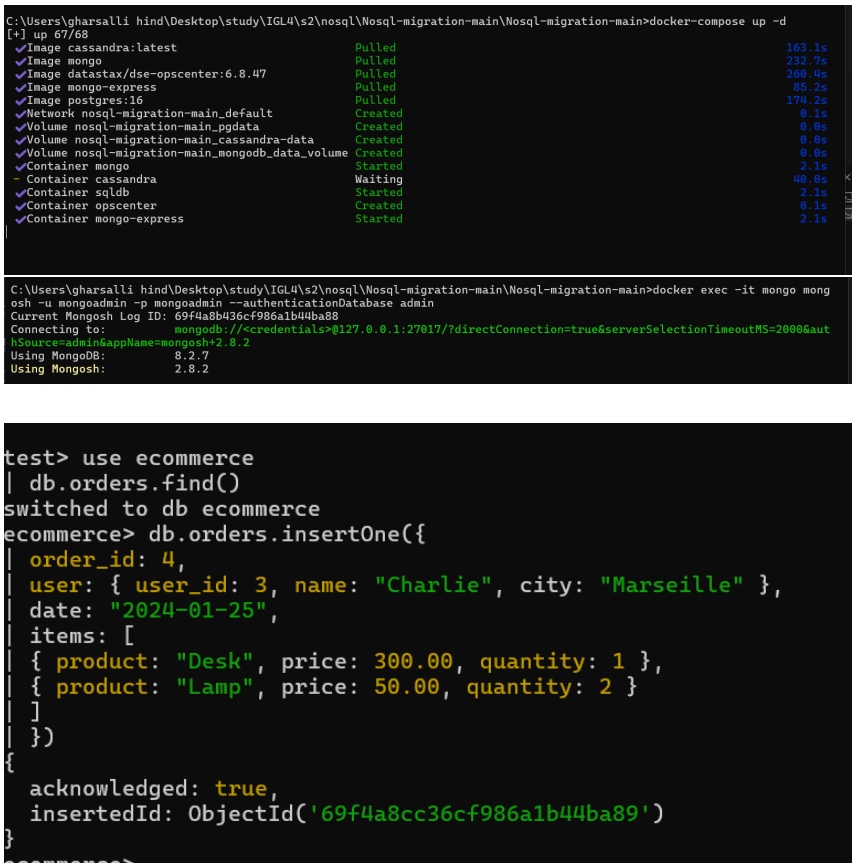


FIGURE 3 – Requêtes et résultats MongoDB (Partie 1)



FIGURE 4 – Requêtes et résultats MongoDB (Partie 2)

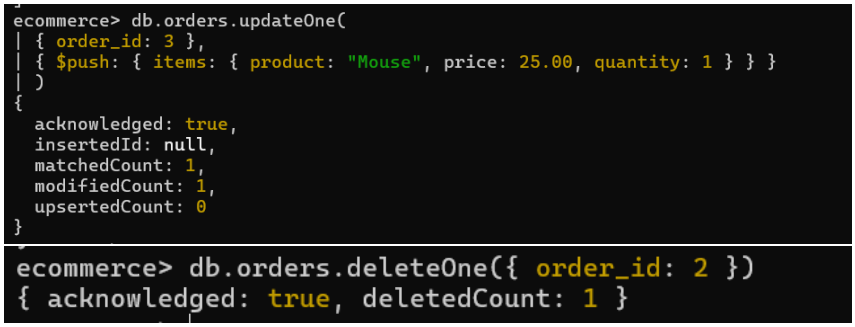


FIGURE 5 – Requêtes et résultats MongoDB (Partie 3)

9.3 Captures d'écran : Cassandra

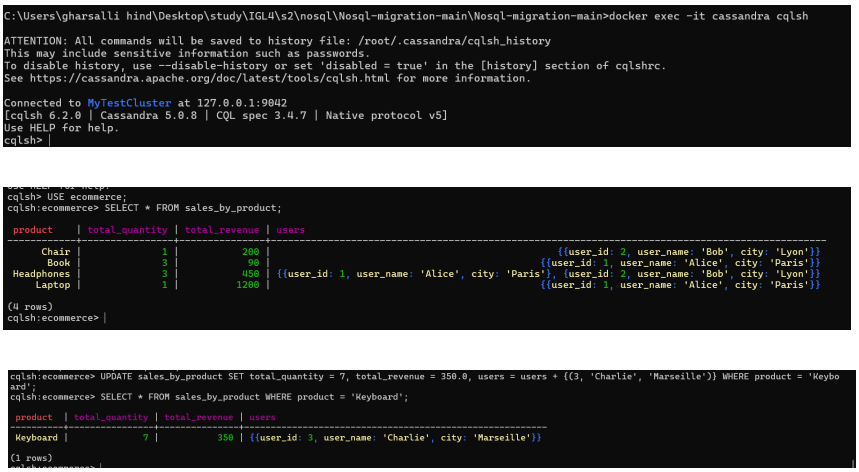


FIGURE 6 – Exécution des requêtes sous Cassandra